LABORATÓRIO 6 – SÍNTESE STANDARD-CELLS UTILIZANDO CADENCE (28 nm)

Prof. Fernando Gehm Moraes – Revisão: 24/outubro/2026

Objetivo deste laboratório:

Compreender o fluxo de projeto *standard cells*
Simulação RTL, simulação com *netlist*, simulação com anotação de atraso de roteamento
Executar a síntese lógica e física

ESTE LABORATÓRIO NÃO FUNCIONA NO onedrive!

wget https://fgmoraes.github.io/microel/lab6.zip (só executar 1 vez!)
unzip lab6.zip
cd lab6
source /soft64/source_gaph

Sigam o procedimento do LAB4: https://fgmoraes.github.io/microel/lab4_layout_Inversor.pdf

Arquivos do projeto (nanoCPU)

Abaixo está a estrutura da distribuição, a qual contém três diretórios principais:

```
.
|-- limpa                          // script de limpeza do laboratório
|-- rtl
|   `-- nanoCPU.sv                 // código SystemVerilog do circuito -  nanoCPU
|-- sim
|   |-- rtl
|   |   |-- file_list.f            // script de simulação RTL
|   |   |-- wave.tcl
|   |   `-- wave.tcl.svcf
|   |-- sdf
|   |   |-- file_list.f            // script de simulação com atraso de fios
|   |   `-- sdf_cmd.cmd
|   |-- synth
|   |   `-- file_list.f            // script de simulação com o netlist da síntese física
|   `-- tb
|       `-- nanoTB.sv              // test bench
`-- synthesis
    |-- comandos_genus.tcl              // script de síntese lógica
    |-- comandos_innovus.tcl       // script de síntese física
    |-- constraint
    |   |-- load.tcl               // parâmetros para a síntese lógica (tecnologia de 28 nm)
    |   `-- restrictions.sdc       // restrições de temporização
    `-- physical // scripts parciais para a síntese física
        |-- 1_init.tcl             // primeira linha aponta para o arquivo de configuração do netlist
        |-- 2_power_plan.tcl       // definição do roteamento de alimentação e polarização das células
        |-- 3_pin_clock.tcl        // posicionamento dos pinos de E/S e geração da árvore de clock
        |-- 4_nano_route.tcl       // roteamento
        `-- 5_fillers_reports.tcl // inserção das células de preenchimento
```

Após executar o laboratório, pesquise e responda (entregar as respostas no dia da segunda avaliação – P2):

1. Qual a finalidade das três simulações executadas no fluxo *standard-cells* (RTL, *netlist*, com SDF)?

2. Qual a função dos arquivos com extensão **LEF**, e qual o significado desta sigla?
   Qual a função dos arquivos com extensão **LIB**, e qual o significado desta sigla?

3. Modifique a frequência utilizada na síntese lógica (`constraint/restrictions.sdc`) e preencha a tabela abaixo:

Frequência	N# de portas lógicas	Slack (ps)
250 MHz		
500 MHz		
1 GHz		
1.5 GHz		

   Com o aumento da frequência, como se comporta o número de portas lógicas (aumenta ou diminui? Por quê?). Com o aumento da frequência, como se comporta o slack time (aumenta ou diminui? Por quê?). E o tempo de síntese?

4. A síntese lógica utiliza três comandos de síntese: `syn_generic` / `syn_map` / `syn_opt`. Qual o objetivo de cada comando? (dica para iniciar a resposta: `man syn_generic` ou `help syn_generic` no terminal do genus).

5. No arquivo *load.tcl* (síntese lógica) há o comando para ativar o modo PLE (*physical layout estimator*):

```
#Set PLE
set_db interconnect_mode ple
```

   Execute ao final da síntese lógica: report_ple. O que ele indica? Porque é importante ativar o modo PLE durante a síntese lógica?

6. Modifique o arquivo de síntese lógica, de forma que os comandos de síntese sejam como abaixo (incluindo a definição do qrc_tech_file). Preencher a tabela abaixo e discutir os novos resultados. Obs.: no Genus 23.14 (módulo ddi) a opção -incremental do syn_opt foi removida (erro SYNTH-29), por isso utiliza-se apenas syn_opt.

```
set_db qrc_tech_file /soft64/design-kits/stm/28nm-
cmos28lp_42/CadenceTechnoKit_cmos028_6U1x_2U2x_2T8x_LB@4.2.1/QRC_TECHFILE/nominal.tech

syn_generic -create_floorplan -physical
syn_map -physical
syn_opt
```

Frequência	N# de portas lógicas	Slack (ps)
500 MHz		
1.0 GHz		

7. Porque na síntese física a geração do *floorplan* deve-se ter uma densidade inferior a 100%, como no comando utilizado neste laboratório?

```
##Generating square floorplan (1) with 85% of density (0.85) with 3um
margins
create_floorplan -site CORE12T -core_density_size 1 0.85 3 3 3 3
```

8. Qual a função da etapa *clock tree synthesis* na síntese física (comando clock_opt_design)? Por que esta etapa é importante no fluxo de projeto?

9. Qual a função das *filler cells* no projeto físico?

10. Qual a tendência do *slack time* após a síntese física? Explicar este comportamento.

ETAPA 1 – Simulação RTL com o simulador *xrun*

Configurar o ambiente e ir para o diretório de simulação RTL:

```
module load xcelium
cd nanoCPU_28/sim/rtl
```

Observar o script de simulação **file_list.f** :

```
-smartorder -work work -top tb -notimingchecks -gui -access +rw
../../rtl/nanoCPU.sv
../tb/nanoTB.sv
```

onde:

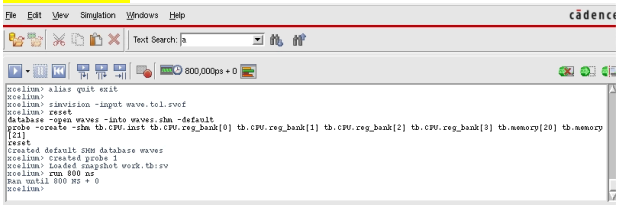
- -smartorder – indica que o compilador deve reconhecer a ordem hierárquica das descrições fornecidas
- -work – define o nome da biblioteca onde serão armazenados os módulos compilados
- -top – topo da hierarquia do projeto (*tb* – módulo do test bench)
- -notimingchecks – desabilita verificações de timing
- -gui – habilita modo gráfico
- -access +rw – acesso aos sinais internos do circuito para exibição

Executar o seguinte comando para inicializar a simulação: **xrun -f file_list.f -input wave.tcl**

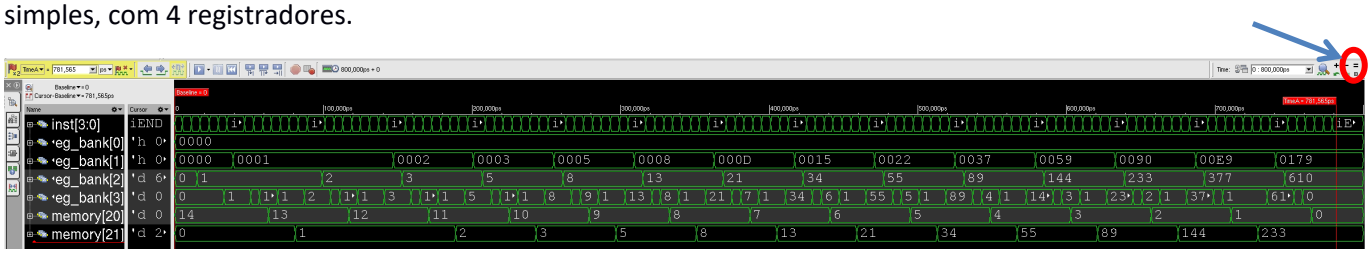
A ferramenta *xrun* irá compilar e elaborar o projeto. A interface do simulador é aberta.

No terminal executar por 800 ns:

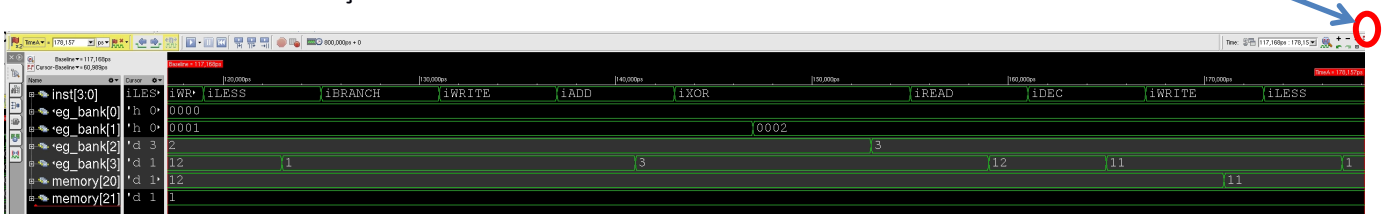
```
reset
run 800 ns
```



Clicar no '=' para zoom. Observar que em **memória[20]** temos um contador que vai de 14 a 0, e em **memória[21]** os primeiros 14 elementos da série de Fibonacci. Estamos simulando uma CPU muito simples, com 4 registradores.

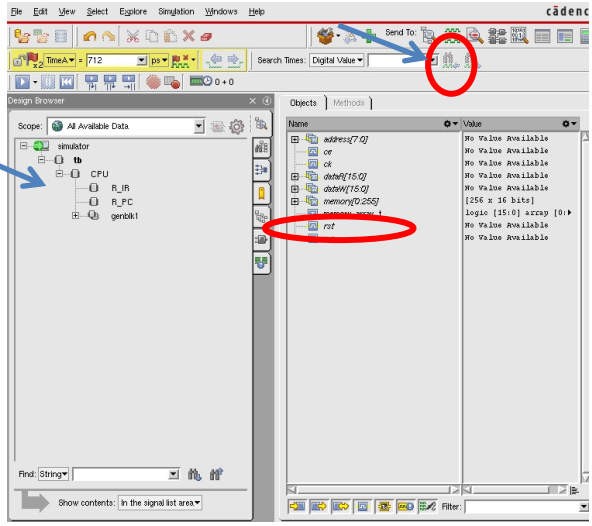


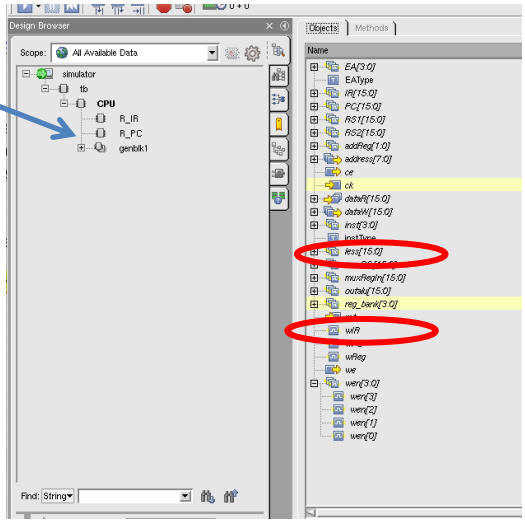
Com as barras verticais de "Baseline" e "TimeA" fazer um zoom no símbolo mais à direita das opções de zoom. Observam-se assim as instruções sendo executadas.



Para sair, menu *File* → *Exit SimVision*
Ao sair do simulador recomenda-se apagar arquivos temporários: **xrun -clean**

O **wave.tcl** é gerado selecionando-se sinais do *top* (**tb** - *memory[20] e memory[21]*) e na CPU os sinais *CPU/reg_bank[3:0]*. Uma vez selecionados salvar o script **wave.tcl**.





ETAPA 2 - Síntese Lógica

Para a síntese lógica é utilizada a ferramenta *genus* da CADENCE. **Não** executar o *genus* com a opção '&', pois a ferramenta tem um *shell* interno. Configurar o ambiente e ir para o diretório de síntese:

```
cd ../../synthesis
module load ddi
genus -gui
```

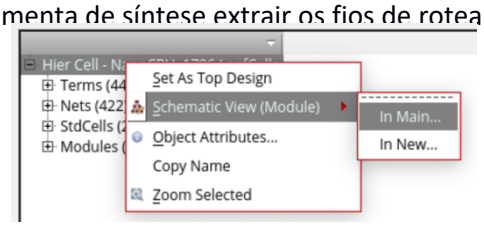
Na interface gráfica poderão ser acompanhadas as respostas dos comandos inseridos no *shell* do *genus*. Os comandos necessários para a correta síntese do projeto estão disponíveis no arquivo *"comandos_genus.tcl"* e deverão ser inseridos sequencialmente no *shell* do *genus*.

A lista de comandos está dividida em **5 grupos distintos** (copie e cole os comandos no *shell* do *genus*).

1. Configuração do ambiente de síntese e compilação do projeto

Abrir o arquivo *load.tcl*. Este é o arquivo que define os LIB e LEF files, e definições gerais para a síntese lógica. Usaremos a biblioteca 28 nm da STMicroelectronics.

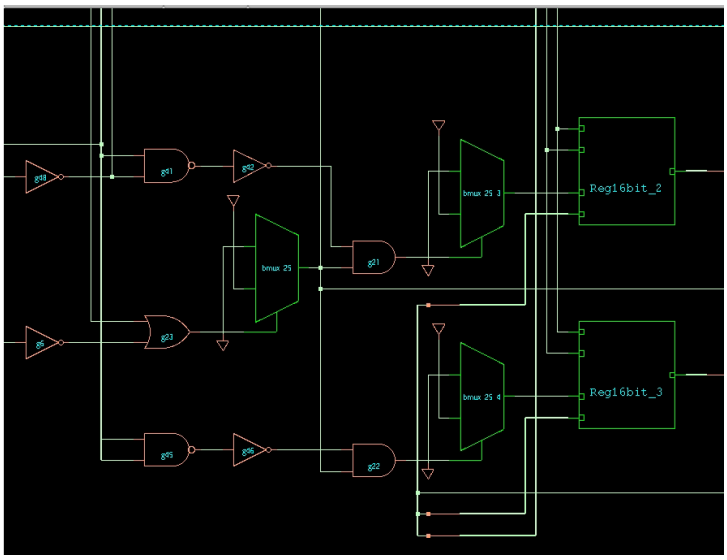
A *captable* modela as interconexões; a condição operacional (processo, tensão e temperatura) é definida pelos arquivos LIB. A *captable* é um arquivo que contém valores de resistência e capacitância usadas para modelar as interconexões do projeto. Essa informação é usada quando a ferramenta de síntese extrair os fios de roteamento, para realizar análises de *timing* e *power*.



Digite os quatro comandos abaixo no *shell* do *genus*:

```
include ./constraint/load.tcl
read_hdl -sv nanoCPU.sv
elaborate NanoCPU
set_db [current_design] .dft_dont_scan true
```

O resultado após executar esse bloco de comandos é dado na janela gráfica do *genus*. Navegar pelo visualizador de esquemáticos. Conforme pode ser observado, o projeto foi elaborado para portas lógicas genéricas (ainda não mapeadas para a tecnologia), como *ands*, *ors*, etc. Selecionar "*schematic view (Module)* → *in Main*". **Faça um zoom em uma região para visualizar as portas lógicas.**



2. Restrições do projeto

Abrir o arquivo de restrições "./constraint/restrictions.sdc" e observar seus comandos.

- Os dois primeiros comandos definem variáveis internas da ferramenta.
- O comando *create_clock* define quem é o clock do circuito e o período desejado **(2.0 ns)**
- O comando *set_false_path* evita que o reset seja utilizado na análise de atraso
- Os comandos *set_input_delay / set_output_delay* definem o atraso externo considerado nas entradas e saídas do circuito (30 ps), em relação ao clock
- O comando *set_load* define a carga nas saídas do circuito

Com essas informações, a ferramenta de síntese pode escolher o ganho/tamanho das células que serão instanciadas no projeto. No *shell* do *genus* digite o comando do segundo grupo:

```
read_sdc ./constraint/restrictions.sdc
```

Como resultado desse comando, deve-se obter a seguinte saída, a qual indica que as *constraints* foram geradas corretamente.

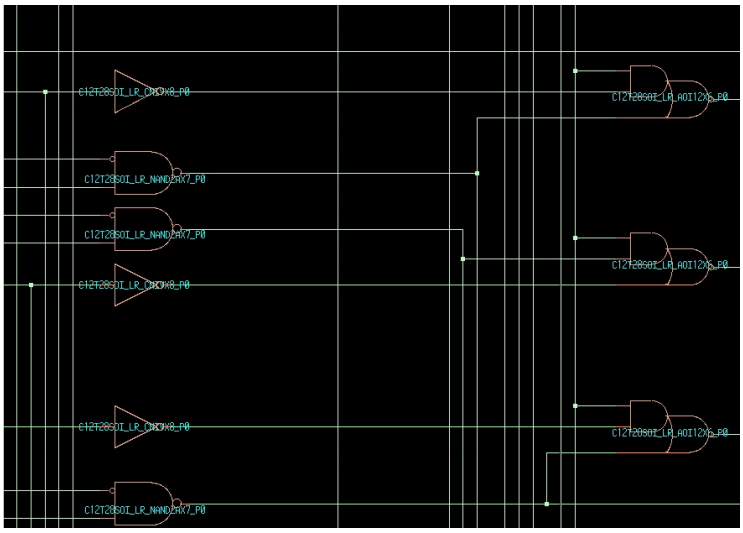
```
Statistics for commands executed by read_sdc:
 "all_inputs"              - successful      1 , failed      0 (runtime  0.00)
 "all_outputs"             - successful      2 , failed      0 (runtime  0.00)
 "create_clock"            - successful      1 , failed      0 (runtime  0.00)
 "get_ports"               - successful      2 , failed      0 (runtime  0.01)
 "set_false_path"          - successful      1 , failed      0 (runtime  0.00)
 "set_input_delay"         - successful      1 , failed      0 (runtime  0.01)
 "set_load"                - successful      1 , failed      0 (runtime  0.00)
 "set_output_delay"        - successful      1 , failed      0 (runtime  0.00)
 "set_units"               - successful      1 , failed      0 (runtime  0.00)
```

3. Síntese lógica otimizada

No *shell* do *genus* digite os comandos abaixo para a realização da síntese lógica:

```
syn_generic
syn_map
syn_opt
```

Esse passo realiza a síntese **otimizada** para o projeto, otimizando o projeto elaborado, para a biblioteca de células de 28 nm.



4. Relatórios

report_area: a coluna "*Instance*" indica o circuito, a coluna "*cell count*" o número de células utilizadas, a coluna "*cell area*" a área dessas células e a coluna "*net area*" uma estimativa da área de fios que será necessária. Notar que neste circuito foram utilizadas 658 células (portas lógicas).

Instance	Module	Cell-Count	Cell-Area	Net-Area	Total-Area
NanoCPU	NA	658	1060.147	425.945	1486.092
R_IR	Reg16bit_98	15	62.016	0.000	62.016
R_PC	Reg16bit_97	9	35.578	0.000	35.578
genblk1_0.reg_inst	Reg16bit	17	70.829	0.000	70.829
genblk1_1.reg_inst	Reg16bit_101	17	70.829	0.000	70.829
genblk1_2.reg_inst	Reg16bit_100	17	70.829	0.000	70.829
genblk1_3.reg_inst	Reg16bit_99	17	70.829	0.000	70.829

report_gates: apresenta todas as portas lógicas utilizadas no projeto.

```
  ...
C12T28SOI_LR_PAO2X8_P0              8     9.139    C28SOI_SC_12_CORE_LR
C12T28SOI_LR_SDFPRQX8_P0           86   378.950    C28SOI_SC_12_CORE_LR
C12T28SOI_LR_XOR3X8_P0              8    18.278    C28SOI_SC_12_CORE_LR
-------------------------------------------------------------------------
total                             658  1060.147
```

Library	Instances	Area	Instances %
C28SOI_SC_12_CLK_LR	34	11.424	5.2
C28SOI_SC_12_CORE_LR	624	1048.723	94.8

Type	Instances	Area	Area %
sequential	90	393.965	37.2
inverter	38	12.403	1.2
buffer	1	0.653	0.1
logic	529	653.126	61.6
physical_cells	0	0.000	0.0
total	658	1060.147	100.0

report_timing: informa o atraso de cada célula no caminho crítico, e principalmente se a síntese atendeu à restrição de timing. Dado que o *clock* é de 2000 ps, há uma sobra de 384 ps (slack) – destacado em amarelo. Observar que temos o tempo de setup de 132 ps, o atraso CP-Q de 122 ps, e depois o atraso combinacional.

```
Path 1: MET (384 ps) Setup Check with Pin genblk1_3.reg_inst/Q_reg_15/CP->TI
           Group: ck
      Startpoint: (R) R_IR/Q_reg_1/CP
           Clock: (R) ck
        Endpoint: (F) genblk1_3.reg_inst/Q_reg_15/TI
           Clock: (R) ck

                     Capture       Launch
       Clock Edge:+     2000            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=    2000            0
```

```
            Setup:-      132
     Required Time:=     1868
      Launch Clock:-        0
         Data Path:-     1485
             Slack:=      384
```

```
#-------------------------------------------------------------------------------------------------------------------------------
#         Timing Point           Flags    Arc   Edge           Cell             Fanout Load Trans Delay Arrival Instance
#                                                                                    (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------------------------------------------
  R_IR/Q_reg_1/CP                -       -      R     (arrival)                    90    -     0     0       0    (-,-)
  R_IR/Q_reg_1/Q                 -       CP->Q  R     C12T28SOI_LR_SDFPRQX8_P0     18 37.5   143   122     122    (-,-)
  g4831__6161/Z                  -       S0->Z  F     C12T28SOI_LR_MUX41X8_P0      14 28.4    78   169     291    (-,-)
  mul_98_34_g3106/Z              -       A->Z   R     C12T28SOI_LR_CNIVX8_P10       9 20.7   108   110     401    (-,-)
  mul_98_34_g3056__4319/Z        -       B->Z   R     C12T28SOI_LRS_XOR2X6_P0      13 20.8   175   136     536    (-,-)
  mul_98_34_g2996__6783/Z        -       B->Z   F     C12T28SOI_LR_NAND2X7_P0      11 18.1   107   126     663    (-,-)
  mul_98_34_g2968__3680/Z        -       B->Z   R     C12T28SOI_LR_OAI22X5_P0       1  4.5    69    82     745    (-,-)
  mul_98_34_g2886__2802/S0       -       A0->S0 F     C12T28SOI_LR_FA1X8_P0         1  4.3    18    78     823    (-,-)
  mul_98_34_g2864__1881/CO       -       B0->CO F     C12T28SOI_LR_FA1X8_P0         1  3.8    19    51     874    (-,-)
  mul_98_34_g2859__1705/CO       -       CI->CO F     C12T28SOI_LR_FA1X8_P0         1  3.8    19    50     924    (-,-)
  mul_98_34_g2857__1617/CO       -       CI->CO F     C12T28SOI_LR_FA1X8_P0         1  3.8    19    50     974    (-,-)
…
  mul_98_34_g2852__4319/CO       -       CI->CO F     C12T28SOI_LR_FA1X8_P0         1  3.8    19    50    1224    (-,-)
  mul_98_34_g2851__6260/CO       -       CI->CO F     C12T28SOI_LR_FA1X8_P0         1  3.8    19    50    1274    (-,-)
  mul_98_34_g2850__5107/CO       -       CI->CO F     C12T28SOI_LR_FA1X8_P0         1  3.0    17    48    1322    (-,-)
  mul_98_34_g2849__2398/Z        -       C->Z   F     C12T28SOI_LR_XOR3X8_P0        1  2.0    17    57    1379    (-,-)
  g4686__5526/Z                  -       S0->Z  F     C12T28SOI_LR_MX41X7_P0        2  3.3    24    58    1436    (-,-)
  g4659__6783/Z                  -       D0->Z  F     C12T28SOI_LR_MUX21X8_P0       4  4.3    20    48    1485    (-,-)
  genblk1_3.reg_inst/Q_reg_15/TI <<<     -      F     C12T28SOI_LR_SDFPRQX8_P0      4    -     -     0    1485    (-,-)
#-------------------------------------------------------------------------------------------------------------------------------
```

report_power -unit mW

```
  ----------------------------------------------------------------------------
    Category          Leakage      Internal     Switching        Total     Row%
  ----------------------------------------------------------------------------
      memory      0.00000e+00   0.00000e+00   0.00000e+00   0.00000e+00   0.00%
    register      2.53746e-04   2.41565e-01   2.29050e-02   2.64724e-01  34.02%
       latch      0.00000e+00   0.00000e+00   0.00000e+00   0.00000e+00   0.00%
       logic      6.36265e-04   1.89275e-01   2.82936e-01   4.72848e-01  60.77%
        bbox      0.00000e+00   0.00000e+00   0.00000e+00   0.00000e+00   0.00%
       clock      0.00000e+00   0.00000e+00   4.05000e-02   4.05000e-02   5.21%
         pad      0.00000e+00   0.00000e+00   0.00000e+00   0.00000e+00   0.00%
          pm      0.00000e+00   0.00000e+00   0.00000e+00   0.00000e+00   0.00%
  ----------------------------------------------------------------------------
    Subtotal      8.90011e-04   4.30841e-01   3.46341e-01   7.78072e-01 100.00%
  Percentage             0.11%        55.37%        44.51%       100.00% 100.00%
       ----------------------------------------------------------------------------
```

5. Exportação para a síntese física

write_netlist [current_design] > nano.v

write_db -common -all_root_attributes nanoCPU_genus.db

Para sair do *genus* digite **exit**

Para executar via script: **genus -f comandos_genus.tcl**

Ao executar via script, os relatórios são gravados no arquivo: **nano.txt**.

ETAPA 3 - Simulação pós síntese com o netlist nano.v

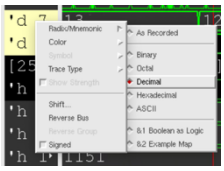
Deve-se garantir que o *netlist*, gerado no passo anterior, implementa a funcionalidade desejada. Para tanto, ir para o ambiente de simulação pós-síntese:

```
module purge
module load xcelium
cd ../sim/synth
```

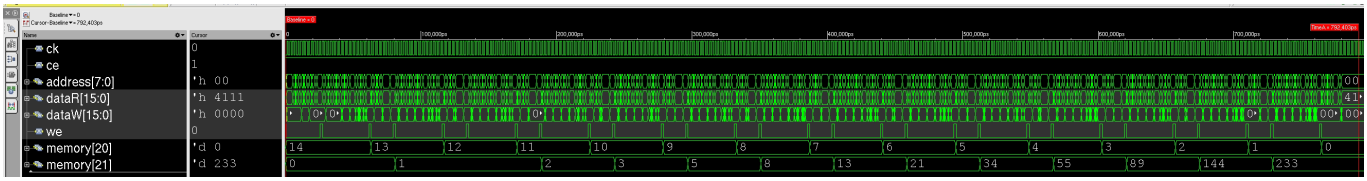
Observar o *netlist* gerado: **more ../../synthesis/nano.v**. Este *netlist*, com 1.815 linhas, corresponde ao circuito original, mapeado para as portas lógicas da tecnologia.

O *script* desse ambiente é similar ao de verificação RTL, porém agora também será compilada a descrição funcional das bibliotecas utilizadas. Isso é necessário pois o *netlist* contém as células específicas da tecnologia. Executar: **more file_list.f**

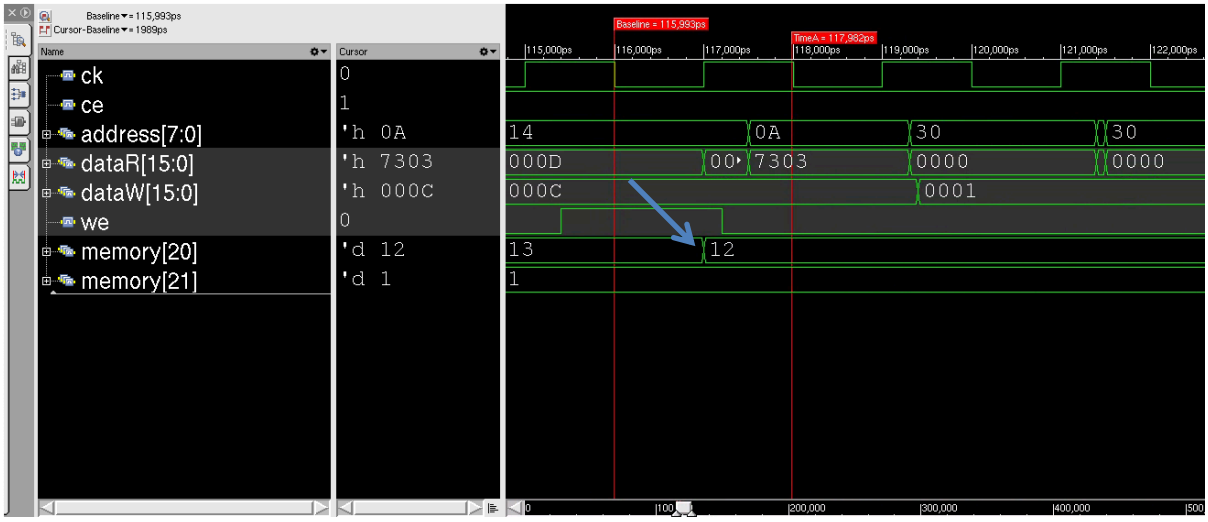
```
-smartorder -work work -top tb -gui -access +rw
/soft64/design-kits/stm/28nm-cmos28fdsoi_25d/C28SOI_SC_12_CLK_LR@2.1@20130621.0/behaviour/verilog/C28SOI_SC_12_CLK_LR.v
/soft64/design-kits/stm/28nm-cmos28fdsoi_25d/C28SOI_SC_12_CORE_LR@2.0@20130411.0/behaviour/verilog/C28SOI_SC_12_CORE_LR.v
../../synthesis/nano.v
../tb/nanoTB.sv
```



Executar o comando **xrun -f file_list.f** - enviar os sinais do top para uma *waveform* e simular o circuito por 800 ns (reset; run 800ns).



Fazer um zoom ao redor do valor de memory[20]=12. Observar que agora temos atrasos em relação ao sinal ck, dado que estamos simulando o *netlist* no nível de portas lógicas, e não mais o projeto no nível RTL (SystemVerilog).



A importância desta simulação é demonstrar que o netlist gerado pela síntese preserva a funcionalidade da descrição RTL, ou seja, que o circuito opera como o esperado no nível de portas lógicas.

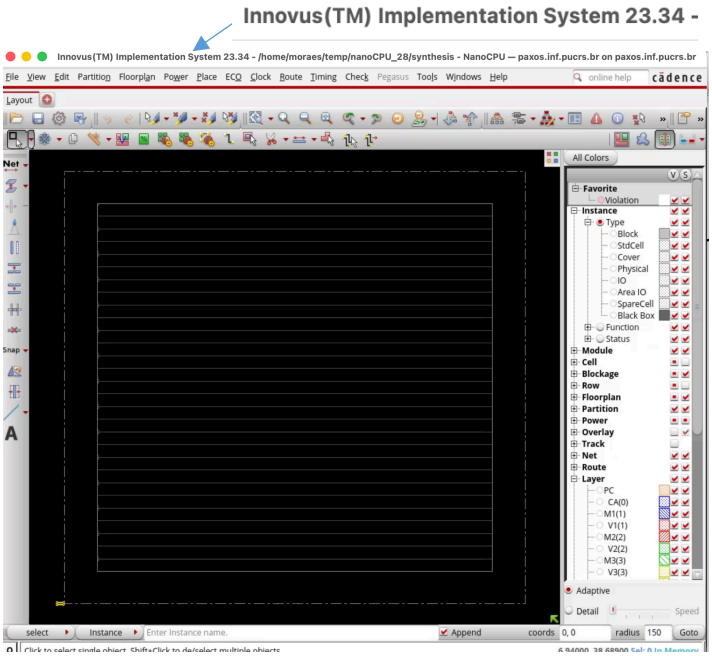
Ao sair do simulador recomenda-se apagar arquivos temporários: **xrun -clean**

ETAPA 4 – SÍNTESE FÍSICA

Uma vez que a síntese lógica do projeto foi validada, deve ser feita a síntese física. Para isto iremos utilizar os arquivos gerados na ferramenta anterior (*genus*) e a ferramenta *innovus* da CADENCE:

```
module load ddi
 cd ../../synthesis
innovus -stylus
```

source physical/1_init.tcl – carrega a configuração inicial da síntese física (comando a ser digitado no *shell* do *innovus*).



Abrir o arquivo *physical/1_init.tcl* e entender os comandos passados para o *innovus*. Dentre os comandos, os 2 principais são:

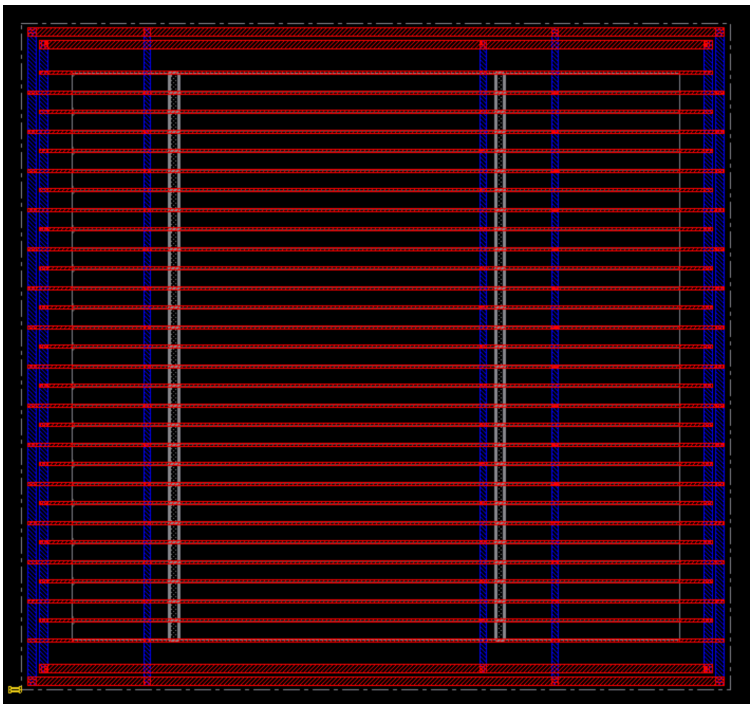
```
##Load the circuit configuration from genus
read_db nanoCPU_genus.db

##Generating square floorplan (1) with 85% of density (0.85) with 3um margins
create_floorplan –site CORE12T –core_density_size 1 0.85 3 3 3 3
```

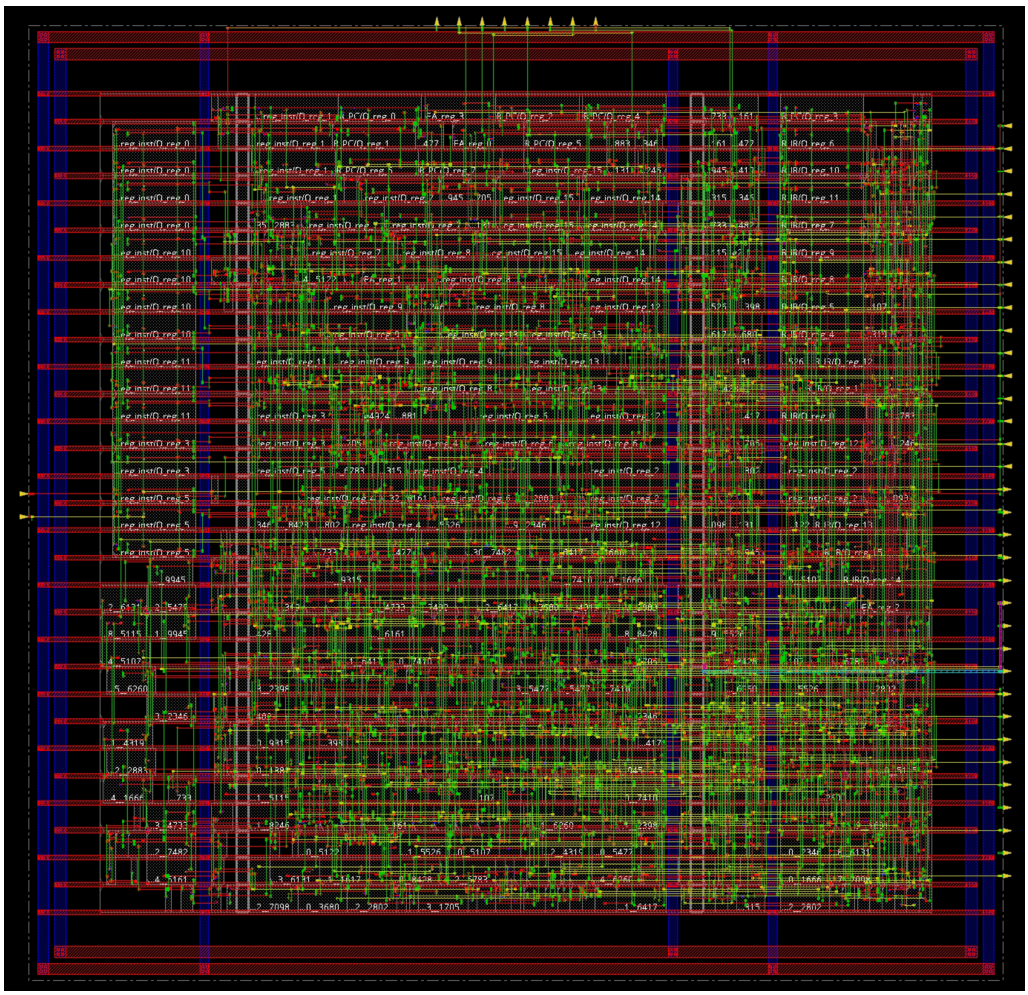
source physical/2_power_plan.tcl – carrega a configuração de *power planning*

Abrir o arquivo ***2_power_plan.tcl***. Notar que foi gerado um anel e linhas de alimentação, que serão utilizadas para posicionar as células lado a lado (***add_rings***). A simetria dessas linhas (mesma altura) facilita o algoritmo de posicionamento e a instanciação das células físicas. As linhas de alimentação correspondem aos retângulos vermelhos (metal 1 - route_special). Além disso, foram posicionadas colunas de *tap cells* (***add_well_taps***). Estas células garantem a polarização dos poços (body bias), já que para essa biblioteca, as células lógicas não possuem conexão com *bulk*. Essas células devem ser posicionadas no máximo 20 µm de distância uma da outra, para garantir polarização dos poços (body bias) (informação obtida na documentação da biblioteca). Também foram inseridos reforços para a alimentação (***add_stripes***), para evitar flutuações nas linhas de alimentação.

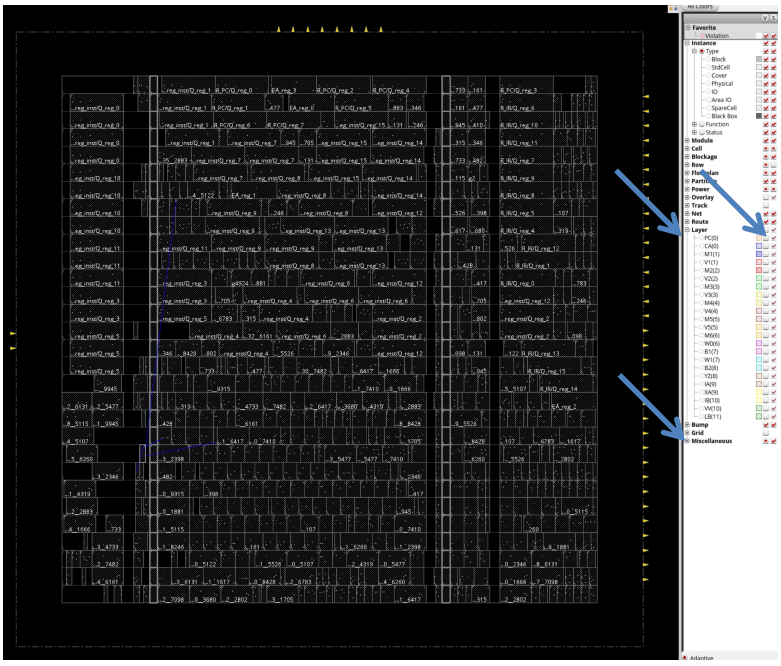
.



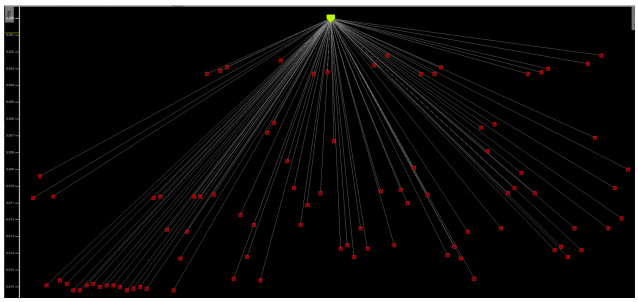
source physical/3_pin_clock.tcl - (demora um pouco) - posiciona as células (place_opt_design), posiciona os pinos na periferia do circuito e realiza a síntese da árvore de *clock*.



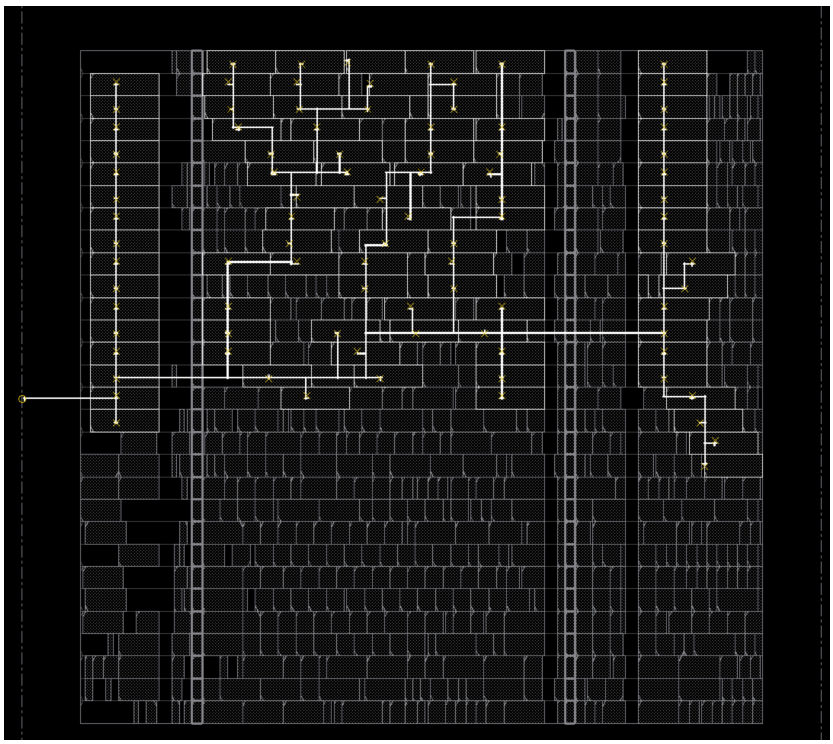
➔ observar que há um menu à direita – se as camadas não aparecerem, selecionar como visível (V) as instâncias, fios, etc. Desmarcar “layer” para visualizar apenas as instâncias das células.



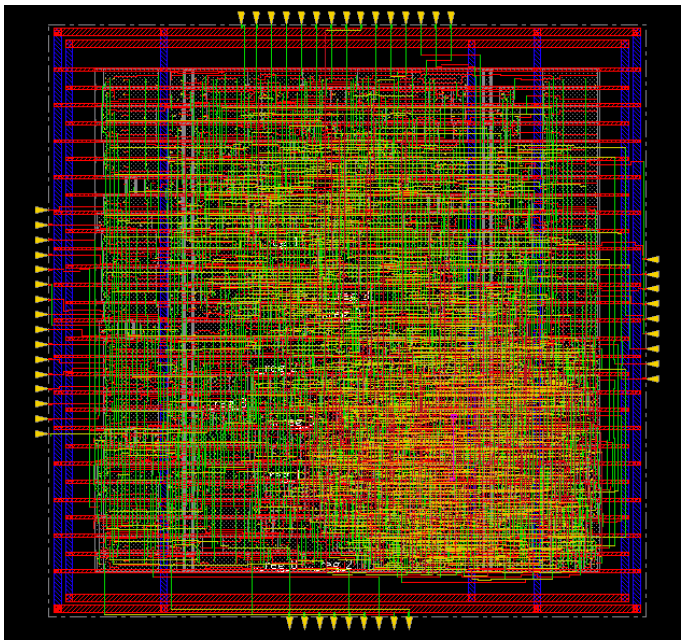
No topo da tela selecionar **clock → CCOpt Clock Debugger** e OK. Esta ação abre uma janela, com os buffers de *clock*:



Na figura acima selecionei todos os sinais de clock e desmarquei "miscellaneous" na palette. O resultado é a visualização da árvore de *clock*, e de todos os registradores alimentados pelo clock



source physical/4_nano_route.tcl - executa o roteamento, ou seja, conecta as células e os pinos de entrada/saída. Marcar para visualizar todos os *layers* e *miscellaneous*.



Este script contém 4 comandos:

```
route_design
route_design –global_detail –wire_opt
set_db timing_analysis_type ocv
opt_design –post_route
```

source physical/5_fillers_reports.tcl. Notar que a figura acima possui "buracos" entre instâncias de células. Tal condição pode representar uma violação nas regras de manufatura, definidas pela *foundry*. Portanto, devem ser incluídas *filler cells*, que preencherão os espaços entre células e garantirão que o projeto não violará regras pelo motivo descrito. O script executa o comando **check_drc**, cujo resultado deve ser:

```
VERIFY DRC ..... Starting Verification
VERIFY DRC ..... Initializing
VERIFY DRC ..... Deleting Existing Violations
VERIFY DRC ..... Creating Sub-Areas
VERIFY DRC ..... Sub-Area: {0.000 0.000 42.160 40.800} 1 of 1
VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

  Verification Complete : 0 Viols.
```

- Os comandos de extração de parasitas servem para gerar o verilog e o SDF para simulação com atraso de roteamento.

report_timing → observar agora que o slack **aumentou** (de 384 ps na síntese lógica para 436 ps): com o posicionamento e o roteamento reais, o innovus otimiza o circuito (place_opt_design, opt_design -post_route), redimensionando células e inserindo buffers

```
Path 1: MET (0.436 ns) Setup Check with Pin genblk1_1.reg_inst/Q_reg_15/CP->TI
              View: default_emulate_view
             Group: ck
       Startpoint: (R) R_IR/Q_reg_1/CP
             Clock: (R) ck
         Endpoint: (F) genblk1_1.reg_inst/Q_reg_15/TI
             Clock: (R) ck

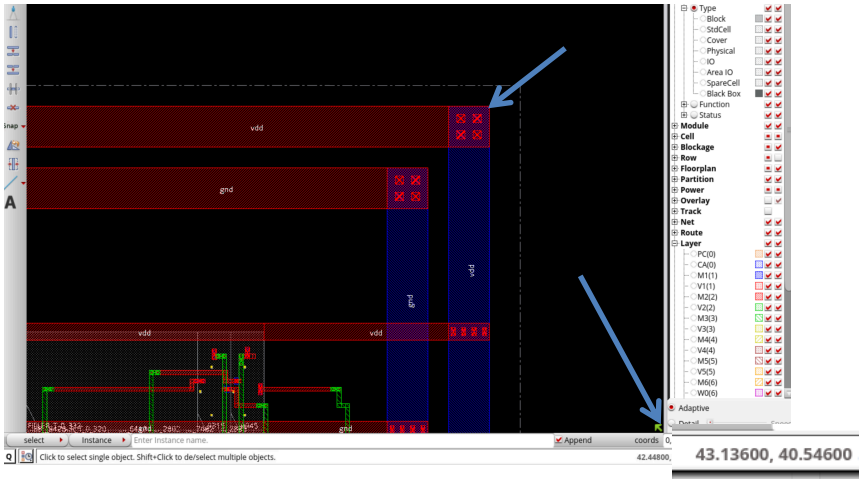
                    Capture       Launch
      Clock Edge:+    2.000        0.000
     Src Latency:+    0.000       -0.010
     Net Latency:+    0.005 (P)    0.010 (P)
         Arrival:=    2.005        0.001

           Setup:-    0.123
   Required Time:=    1.883
    Launch Clock:=    0.001
       Data Path:+    1.446
           Slack:=    0.436
```

report_area

```
   Hinst Name       Module Name          Inst Count          Total Area
-------------------------------------------------------------------------
NanoCPU                                        652              1058.026
```

Observar que esta é a área de células. Posicionar o mouse no canto superior do circuito como abaixo. O circuito tem uma área aproximada de 42,16µm x 40,80 µm, o que resulta em 1720 **µm²**.



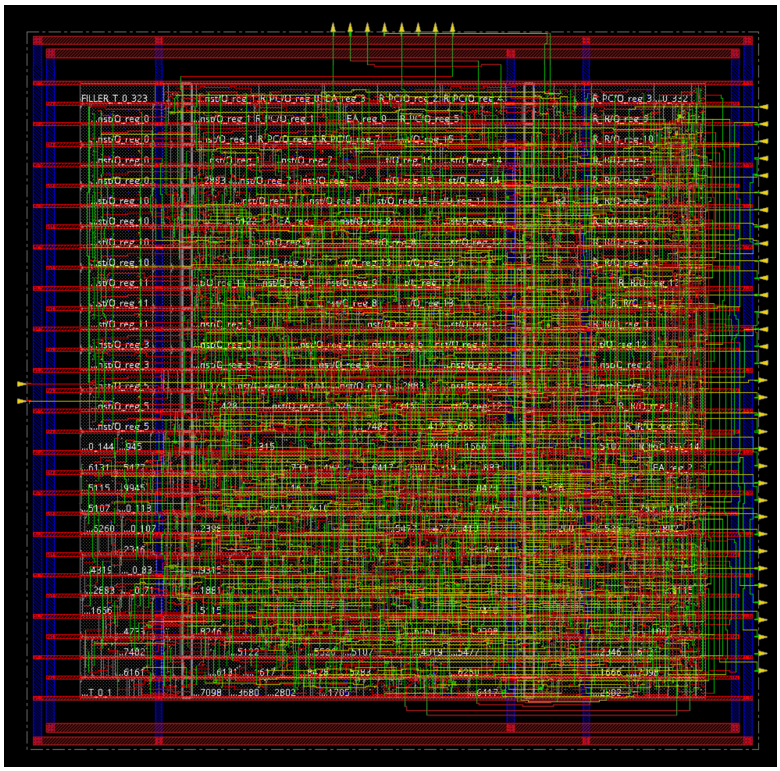
Observar que foi gerado um relatório do projeto, no diretório "*summaryReport*". Executar o seguinte na linha de comando (pode ser dentro do *innovus*):

firefox summaryReport/NanoCPU.main.htm

Neste relatório temos por exemplo a área do *core* (área sem o anel de alimentação) e a área total do circuito, que confere com a medida realizada acima:

Floorplan/Placement Information	
Total area of Standard cells	1249.459 um^2
Total area of Standard cells(Subtracting Physical Cells)	1058.026 um^2
Total area of Core	1249.459 um^2
Total area of Chip	**1720.128 um^2**

Explorar informação como, células instanciadas, área do core, comprimentos de fios, níveis de metal utilizados, etc.



Para sair: exit

ETAPA 5 – SIMULAÇÃO COM ATRASO DE ROTEAMENTO

Neste passo iremos simular o circuito com atraso de portas e fios. O arquivo que contém estes atrasos é o ***nano_pr.sdf***. A descrição *sdf* é um formato padronizado (IEEE 1497), independente da linguagem de descrição de hardware, e significa *Standard Delay Format.*

Executar:

```
module purge
module load xcelium
cd ../sim/sdf                    (pasta de simulação com atraso de roteamento)
```

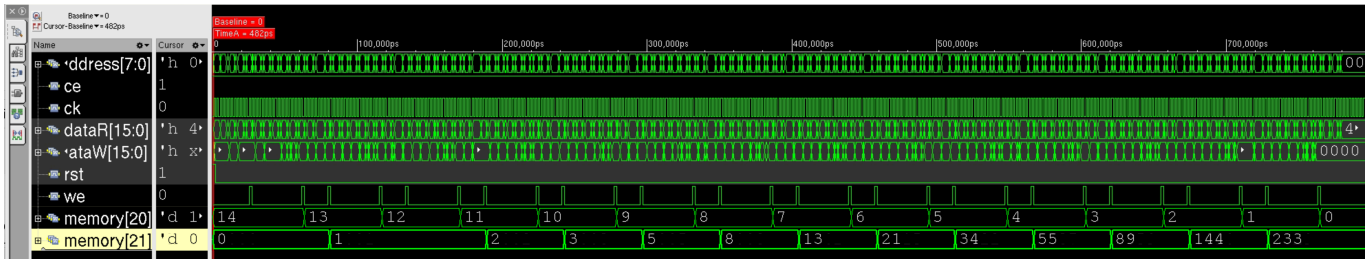
O script para esta simulação é similar ao de verificação pós-síntese, porém agora é dado um parâmetro a mais (*-sdf_cmd_file*), o script de configuração de atraso. Ver **more sdf_cmd.cmd** :

```
SDF_FILE = "../../synthesis/nano_pr.sdf",
LOG_FILE = "./sdf_log.log",
SCOPE = tb.CPU,
MTM_CONTROL = "MAXIMUM",
SCALE_FACTORS = "1.0:1.0:1.0",
SCALE_TYPE = "FROM_MAXIMUM";
```

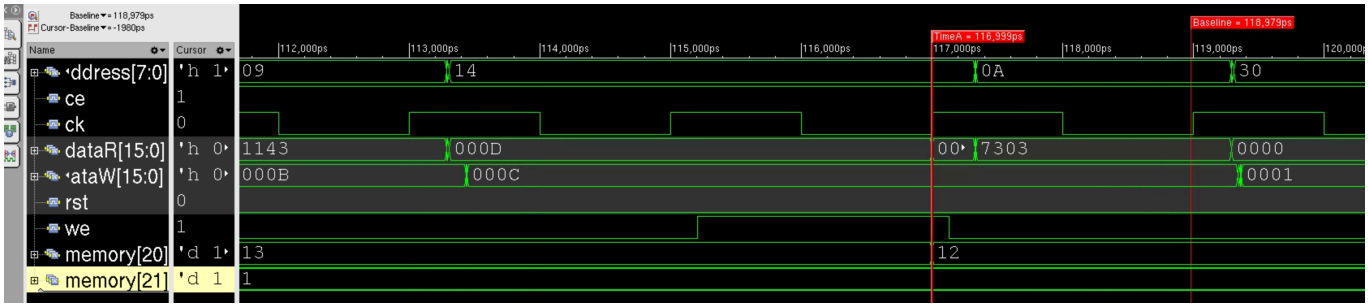
Arquivo: **file_list.f**

```
-smartorder -work work -top tb -gui -access +rw -maxdelays -sdf_cmd_file sdf_cmd.cmd
/soft64/design-kits/stm/28nm-cmos28fdsoi_25d/…/behaviour/verilog/C28SOI_SC_12_CLK_LR.v
/soft64/design-kits/stm/28nm-cmos28fdsoi_25d/…/behaviour/verilog/C28SOI_SC_12_CORE_LR.v
../../synthesis/nano_pr.v
../tb/nanoTB.sv
```

Executar o comando **xrun -f file_list.f** - enviar os sinais do top
para uma *waveform* e simular o circuito por 800 ns (reset; run 800ns).



Visualizando atrasos:



Conferir o sdf_log.log, não deve ter erro neste arquivo.

Observar o arquivo xrun.log (parte transcrita abaixo). Indica que todos os caminhos foram “anotados”.

```
SDF statistics:
    No. of Pathdelays = 8746      No. of Disabled Pathdelays = 0        Annotated = 100.00% (8746/8746)
    No. of Tchecks    = 1056      No. of Disabled Tchecks    = 0        Annotated = 100.00% (1056/1056)
                           Total(T)     Disabled(D)        Annotated(A)      Percentage(A/(T-D))
    Path Delays             8746             0                8746                100.00
         $width              442             0                 442                100.00
        $recrem               90             0                  90                100.00
      $setuphold             524             0                 524                100.00
Building instance overlay tables: ..................... Done
```

Abra o arquivo nano_pr.txt (gerado quando a síntese física é executada via script comandos_innovus.tcl; com os scripts parciais os relatórios aparecem no terminal) e observe o caminho crítico. É possível navegar na hierarquia do projeto e visualizar a saída Q do IR, assim como a entrada TI do Q_reg_15

```
             Slack:=    0.436

#-------------------------------------------------------------------------------------------------------------------
--
# Timing Point                    Flags  Arc     Edge  Cell                    Fanout  Trans   Delay  Arrival
#                                                                                        (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------------------------------------
--
  R_IR/Q_reg_1/CP                 -      CP      R     (arrival)                 90  0.020       -    0.001
  R_IR/Q_reg_1/Q                  -      CP->Q   R     C12T28SOI_LR_SDFPRQX8_P0  18  0.020   0.131    0.132
  g4831__6161/Z                   -      S0->Z   F     C12T28SOI_LR_MUX41X8_P0   14  0.140   0.169    0.301
  mul_98_34_g3106/Z               -      A->Z    R     C12T28SOI_LR_CNIVX8_P10    9  0.081   0.108    0.409
  mul_98_34_g3056__4319/Z         -      B->Z    R     C12T28SOI_LRS_X0R2X6_P0   13  0.100   0.127    0.536
  mul_98_34_g2996__6783/Z         -      B->Z    F     C12T28SOI_LR_NAND2X7_P0   11  0.160   0.119    0.655
  mul_98_34_g2968__3680/Z         -      B->Z    R     C12T28SOI_LR_OAI22X5_P0    1  0.102   0.078    0.733
  mul_98_34_g2886__2802/S0        -      A0->S0  F     C12T28SOI_LR_FA1X8_P0      1  0.064   0.074    0.807
  mul_98_34_g2864__1881/CO        -      B0->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.048    0.855
  mul_98_34_g2859__1705/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.017   0.046    0.901
  mul_98_34_g2857__1617/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.047    0.948
  mul_98_34_g2856__3680/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.017   0.047    0.994
  mul_98_34_g2855__6783/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.046    1.040
  mul_98_34_g2854__5526/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.046    1.086
  mul_98_34_g2853__8428/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.046    1.132
  mul_98_34_g2852__4319/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.047    1.179
  mul_98_34_g2851__6260/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.017   0.047    1.226
  mul_98_34_g2850__5107/CO        -      CI->CO  F     C12T28SOI_LR_FA1X8_P0      1  0.016   0.045    1.270
  mul_98_34_g2849__2398/Z         -      C->Z    F     C12T28SOI_LR_XOR3X8_P0     1  0.014   0.061    1.331
  g4686__5526/Z                   -      S0->Z   F     C12T28SOI_LR_MX41X7_P0     2  0.021   0.065    1.396
  g4659__6783/Z                   -      D0->Z   F     C12T28SOI_LR_MUX21X8_P0    4  0.030   0.051    1.447
  genblk1_1.reg_inst/Q_reg_15/TI  -      TI      F     C12T28SOI_LR_SDFPRQX8_P0   4  0.019   0.000    1.447
#-------------------------------------------------------------------------------------------------------------------
```



FINAL DO TUTORIAL